

Technische Internetbasierte Systeme

Auftakt, Ablauf und Projektarbeit

Linus Hoppe

Aalen University

May 19, 2026



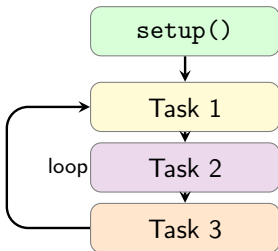
Hochschule Aalen



- ① Rückblick
- ② Grundlagen Wifi
- ③ WLAN-Zugangsdaten
- ④ ESP8266 als Webserver
- ⑤ Asynchroner Webserver



- ➊ Rückblick
- ➋ Grundlagen Wifi
- ➌ WLAN-Zugangsdaten
- ➍ ESP8266 als Webserver
- ➎ Asynchroner Webserver



Wichtige Regel

```
if (now - lastTime >= interval)
```

Wichtige Punkte

- `delay()` blockiert
- `millis()` ermöglicht nicht-blockierende Abläufe
- Zeitvergleiche müssen Rollover-sicher sein
- Watchdog erkennt blockierte Programme
- RTOS trennt Aufgaben in Tasks

Merksatz

Tasks müssen kurz laufen oder CPU-Zeit bewusst freigeben.



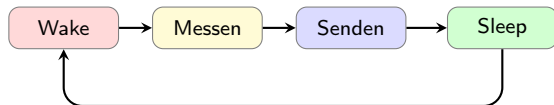
Speicher: Was bleibt erhalten?

Speicher	Nutzung	Erhalten
RAM	Variablen	nein
EEPROM	Konfiguration	ja
LittleFS	Dateien	ja
RTC	kurzer Zustand	teilweise

Merksatz

EEPROM und LittleFS liegen beim ESP meist im Flash.

Energie: Wann ist der ESP aktiv?



- Modem Sleep: WLAN reduziert
- Light Sleep: CPU kann pausieren
- Deep Sleep: Neustart nach Wake-up
- ESP8266: D0/GPIO16 -> RST

Kernaussage

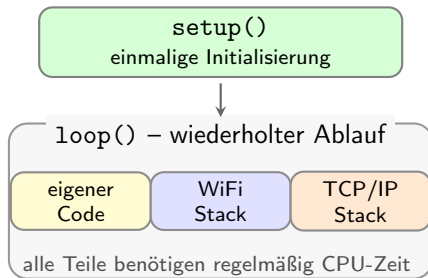
Aktive Zeit kurz halten, Schreibzugriffe bewusst einsetzen.



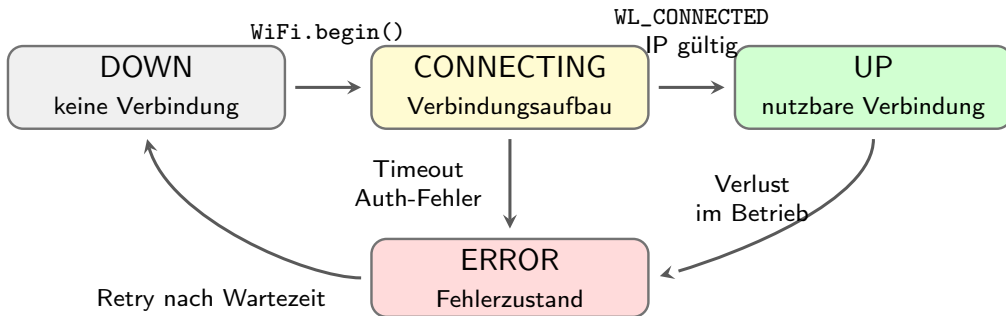
- ① Rückblick
- ② Grundlagen Wifi
- ③ WLAN-Zugangsdaten
- ④ ESP8266 als Webserver
- ⑤ Asynchroner Webserver



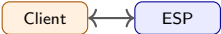
- Der ESP8266 besitzt neben der CPU ein integriertes WiFi-Subsystem.
- `WiFi.begin(...)` startet nur den Verbindungsaufbau.
- WiFi- und TCP/IP-Aufgaben laufen neben dem eigenen Sketch.
- Lange blockierende Abschnitte stören Netzwerk und Reaktionsverhalten.



WiFi-Verbindung als Zustandsautomat





Modus	Rolle im Netzwerk	Typische Verwendung
WIFI_OFF		Funkmodul aus. Sinnvoll für lokale Verarbeitung, Messbetrieb ohne Netzwerk oder Energiesparen.
WIFI_STA		ESP8266 verbindet sich mit einem vorhandenen WLAN. Normalfall für HTTP, MQTT, NTP und Cloud-Anbindung.
WIFI_AP		ESP8266 stellt selbst ein WLAN bereit. Geeignet für Erstkonfiguration oder direkten lokalen Zugriff.
WIFI_AP_STA		Kombinierter Betrieb. Nützlich für Setup, Diagnose oder Fallback, aber komplexer im Verbindungsmanagement.



```
#include <ESP8266WiFi.h>

const char* ssid = "ssid";
const char* password = "password";

void setup() {
    Serial.begin(115200);

    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.print("IP: ");
    Serial.println(WiFi.localIP());
}
```

Eigenschaft

- kurzer Beispielcode
- gut für erste Funktionstests
- einfach beobachtbar über Serial Monitor

Aber

Dieser Code besitzt keine definierte Fehlerstrategie.



```
bool connectWiFi(unsigned long timeoutMs) {
    unsigned long start = millis();

    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
        if (millis() - start >= timeoutMs) {
            return false;
        }
        delay(250); // allow WiFi background tasks
    }

    return true;
}

void setup() {
    if (!connectWiFi(15000)) {
        ESP.deepSleep(60e6); // schalfen gehen
    }

    sendData();
    ESP.deepSleep(60e6); // schalfen gehen
}
```

Problem

- WLAN kann fehlen
- Passwort kann falsch sein
- DHCP kann scheitern
- Signal kann instabil sein

Regel

Nicht unbegrenzt warten.
Verbindungsaufbau bekommt einen
Timeout und einen definierten
Fehlerpfad.



```
#include <ESP8266WiFi.h>

const char* ssid = "ssid";
const char* password = "password";

IPAddress ip(192, 168, 1, 50);
IPAddress gateway(192, 168, 1, 1);
IPAddress subnet(255, 255, 255, 0);
IPAddress dns(192, 168, 1, 1);

void setup() {
  Serial.begin(115200);

  WiFi.mode(WIFI_STA);

  WiFi.config(ip, gateway, subnet, dns);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println(WiFi.localIP());
}
```

Prinzip

- keine IP-Adresse per DHCP
- Adresse wird im Sketch festgelegt
- Gateway ist meist der Router
- DNS wird für Hostnamen benötigt

Wichtig

Die statische IP muss zum Netzwerk passen und darf nicht bereits von einem anderen Gerät verwendet werden.



Einordnung

WL_CONNECTED zeigt nur, dass die WLAN-Verbindung aufgebaut wurde. Für eine nutzbare Netzwerkverbindung müssen zusätzlich IP-Konfiguration und Signalqualität geprüft werden.

Prüfung	Rückgabewert	Bedeutung
WiFi.status()	WL_CONNECTED	WLAN-Verbindung steht. Erst dann sind Netzwerkzugriffe sinnvoll.
WiFi.status()	WL_NO_SSID_AVAIL	Das angegebene WLAN wurde nicht gefunden.
WiFi.status()	WL_CONNECT_FAILED	Verbindungsaufbau fehlgeschlagen, häufig durch falsche Zugangsdaten.
WiFi.status()	WL_DISCONNECTED	Keine aktive WLAN-Verbindung.
WiFi.localIP()	z. B. 192.168.1.42	Lokale IP-Adresse des ESP8266. Ohne gültige IP ist das Gerät nicht sinnvoll adressierbar.
WiFi.gatewayIP()	z. B. 192.168.1.1	Gateway zum lokalen Netzwerk oder Internet.
WiFi.dnsIP()	z. B. 192.168.1.1	DNS-Server für Namensauflösung, z. B. bei HTTP-Requests mit Hostnamen.
WiFi.RSSI()	z. B. -45	Empfangsstärke in dBm. Werte nahe 0 sind stärker, Werte unter etwa -80 meist kritisch.



Bedeutung

- `WiFi.RSSI()` liefert die Empfangsstärke in dBm.
- Der Wert ist negativ.
- Näher an 0 dBm bedeutet stärkeres Signal.
- Schlechter RSSI kann Latenz, Paketverluste und Abbrüche verursachen.

RSSI	Einordnung
≥ -40 dBm	sehr stark
-50 dBm	gut
-60 dBm	brauchbar
-70 dBm	schwach
-80 dBm	instabil
≤ -85 dBm	kritisch

Hinweis

RSSI ist nur ein Diagnosewert. Störungen und Kanalbelegung werden nicht vollständig abgebildet.

Praxisregel

Für stabile Aufbauten möglichst besser als -70 dBm.



- 1 Rückblick
- 2 Grundlagen Wifi
- 3 WLAN-Zugangsdaten**
- 4 ESP8266 als Webserver
- 5 Asynchroner Webserver



- WLAN-Zugangsdaten sollten nicht fest im Hauptprogramm stehen.
- Der Sketch enthält dann keine projektspezifischen Zugangsdaten.
- Zugangsdaten können lokal in eine separate Datei ausgelagert werden.
- Diese Datei wird nicht in die Versionsverwaltung übernommen.

Ziel

Der eigentliche Programmcode bleibt teilbar und versionierbar.

Lokale Zugangsdaten bleiben auf dem jeweiligen Entwicklungsrechner.

Wichtig

Eine ausgelagerte Datei ist keine Verschlüsselung. Sie verhindert nur, dass Passwörter versehentlich im Sketch oder Repository landen.



Ansatz	Prinzip
<code>secrets.h</code>	Zugangsdaten werden beim Kompilieren aus einer lokalen Datei eingebunden.
Konfigurationsdatei	Zugangsdaten liegen als Datei im Flash-Dateisystem und werden beim Start gelesen.
WiFiManager	ESP startet bei Bedarf ein lokales Konfigurationsportal zur Eingabe der Zugangsdaten.
Produktives Provisioning	Zugangsdaten werden über ein dafür vorgesehenes Setup-Verfahren auf das Gerät übertragen.

Begriff

Provisioning bedeutet: Zugangsdaten werden nachträglich auf das Gerät übertragen, ohne den Sketch neu zu flashen.

Variante 1: lokale Datei secrets.h



Datei: secrets.h

```
#pragma once

const char* WIFI_SSID = "your-ssid";
const char* WIFI_PASSWORD = "your-password";
```

Datei: main.ino

```
#include <ESP8266WiFi.h>
#include "secrets.h"

void setup() {
    Serial.begin(115200);

    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}
```

Prinzip

Der Sketch kennt nur die Namen der Konstanten. Die konkreten Zugangsdaten stehen in einer lokalen Datei.



Datei: .gitignore

```
secrets.h
```

Datei: secrets.example.h

```
#pragma once

const char* WIFI_SSID = "your-ssid";
const char* WIFI_PASSWORD = "your-password";
```

#pragma once verhindert, dass diese Header-Datei versehentlich mehrfach eingebunden wird.

Ablauf im Projekt

- 1 secrets.example.h wird versioniert.
- 2 Jeder Entwickler kopiert die Datei zu secrets.h.
- 3 Die eigenen Zugangsdaten werden lokal eingetragen.
- 4 secrets.h bleibt lokal und wird nicht committed.

Kontrolle

Vor dem Commit prüfen, ob keine Zugangsdaten im Repository landen.



```
#if __has_include("secrets.h")
  #include "secrets.h"
#else
  #error "Missing secrets.h. Copy secrets.example.
        h to secrets.h and insert WiFi credentials."
#endif

#include <ESP8266WiFi.h>

void setup() {
  Serial.begin(115200);

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}
```

Bedingte Kompilierung

- `#if` wird vor dem eigentlichen Kompilieren ausgewertet.
- `__has_include()` prüft, ob eine Datei vorhanden ist.
- Fehlt `secrets.h`, bricht `#error` den Build ab.
- Der Fehler wird dadurch früh erkannt, nicht erst beim Start auf dem ESP.

Zweck

Das Projekt kompiliert nur, wenn die lokale Datei mit den Zugangsdaten vorhanden ist. Fehlende Konfiguration wird damit beim Build sichtbar und nicht erst zur Laufzeit auf dem Gerät.



Prinzip

- Zugangsdaten stehen nicht im Sketch.
- Der ESP8266 liest sie beim Start aus LittleFS.
- Änderungen sind möglich, ohne den Sketch neu zu kompilieren.
- Die Datei kann später auch durch ein Portal geschrieben werden.

Grenze

Die Datei ist nicht automatisch verschlüsselt.
Sie trennt Konfiguration von Programmcode.

Beispieldatei: /wifi.conf

```
ssid=Lab-WiFi  
password=secret
```

Ablauf

- ① Datei lokal anlegen
- ② LittleFS-Image erzeugen
- ③ Image in den Flash laden
- ④ Datei beim Start lesen



Problem

Der normale Sketch-Upload überträgt nur die Firmware. Dateien aus data/ werden nicht automatisch mit hochgeladen.

- Konfiguration liegt getrennt vom Programmcode.
- Aus data/ wird ein LittleFS-Image erzeugt.
- Dieses Image wird separat in die Dateisystem-Partition geschrieben.

Merksatz

Firmware-Upload und Dateisystem-Upload sind zwei getrennte Schritte.

Tool: <https://github.com/earlephilhower/arduino-littlefs-upload>

Aufgabe: LittleFS-Uploader installieren



- 1 arduino-littlefs-upload herunterladen.
- 2 VSIX-Datei nach ~/.arduinoIDE/plugins/ kopieren.
- 3 Arduino IDE neu starten.
- 4 Board und Port auswählen.
- 5 Serial Monitor schließen.
- 6 Command Palette öffnen: Ctrl + Shift + P.
- 7 Befehl ausführen: Upload LittleFS to Pico/ESP8266/ESP32.



Hinweis

Bei Upload-Problemen: einmal ein anderes Board auswählen und danach wieder das richtige ESP-Board setzen.



Projektstruktur

```
littleFS_test/  
|-- littleFS_test.ino  
'-- data/  
    '-- wifi.conf
```

Upload-Schritte

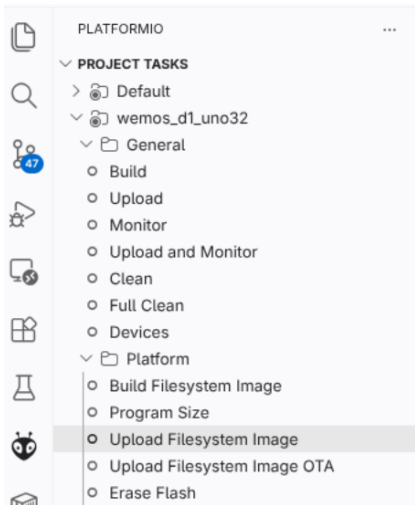
- ① Sketch normal hochladen
- ② LittleFS-Upload ausführen
- ③ ESP neu starten
- ④ Serial Monitor prüfen

Datei: data/wifi.conf

```
ssid=Lab-WiFi  
password=secret
```

Pfad im ESP

Im Sketch wird die Datei als `/wifi.conf` geöffnet. Der Ordner `data/` existiert nur lokal auf dem Rechner.



VS-Code-Workflow

- 1 Upload lädt die Firmware
- 2 Build Filesystem Image erzeugt das Image
- 3 Upload Filesystem Image lädt data/
- 4 Monitor zeigt die serielle Ausgabe

Terminal-Äquivalent

```
pio run -t upload  
pio run -t uploadfs
```



Projektstruktur

```
littleFS_test/  
|-- src/  
|   '-- main.cpp  
|-- platformio.ini  
'-- data/  
    '-- wifi.conf
```

Arduino-IDE-Struktur

```
littleFS_test/  
|-- littleFS_test.ino  
'-- data/  
    '-- wifi.conf
```

Datei: data/wifi.conf

```
ssid=Lab-WiFi  
password=secret
```

Pfad auf dem ESP

Im Code wird die Datei als /wifi.conf geöffnet. Der Ordner data/ existiert nur lokal.



```
#include <Arduino.h>
#include <LittleFS.h>
const char* CONFIG_PATH = "/wifi.conf";

void setup() {
  Serial.begin(115200);
  delay(5000);
  Serial.println("LittleFS upload test");

  // Do not format automatically
  if (!LittleFS.begin(false)) {
    Serial.println("ERROR: LittleFS mount failed");
    return;
  }

  if (!LittleFS.exists(CONFIG_PATH)) {
    Serial.println("ERROR: /wifi.conf not found");
    return;
  }

  File file = LittleFS.open(CONFIG_PATH, "r");

  while (file.available()) {
    Serial.write(file.read());
  }
  file.close();
} void loop(){}
}
```

Was wird geprüft?

- LittleFS wird eingebunden.
- Es wird nicht automatisch formatiert.
- Die Datei /wifi.conf muss bereits hochgeladen sein.
- Der Inhalt wird direkt im Serial Monitor ausgegeben.

Ziel

Nach dem Test ist sichtbar, ob der Dateisystem-Upload funktioniert hat.



Fehlerbild	Prüfen
<code>/wifi.conf not found</code>	Wurde Upload Filesystem Image ausgeführt?
<code>No port specified</code>	Board und Port neu auswählen. Serial Monitor schließen.
Upload startet nicht	Port wird von Serial Monitor oder anderem Programm blockiert.
<code>No module named intelhex</code>	<code>pip install intelhex</code> ausführen, danach <code>pio pkg update</code> .
Datei wird nicht gefunden	Lokal: <code>data/wifi.conf</code> ESP: <code>/wifi.conf</code>

Merksatz

Sketch-Upload lädt den Code. Filesystem-Upload lädt die Dateien aus `data/`.



```
#include <ESP8266WiFi.h>
#include <LittleFS.h>

void setup() {
    Serial.begin(115200);

    if (!LittleFS.begin()) {
        Serial.println("LittleFS failed");
        return;
    }

    WiFiConfig cfg = readWiFiConfig();

    if (cfg.ssid.length() == 0) {
        Serial.println("Missing SSID");
        return;
    }

    WiFi.mode(WIFI_STA);
    WiFi.begin(
        cfg.ssid.c_str(),
        cfg.password.c_str()
    );

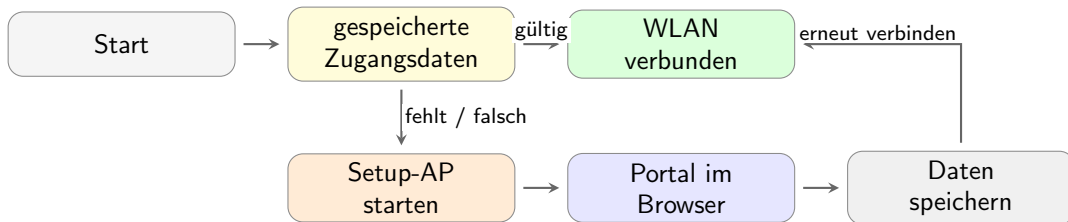
    Serial.println("WiFi started");
}
```

Ablauf

- 1 LittleFS starten
- 2 Datei /wifi.conf öffnen
- 3 Werte einlesen
- 4 SSID prüfen
- 5 WiFi-Verbindung starten

Einordnung

Diese Variante trennt
Programmcode und Konfiguration.



Prinzip

WiFiManager versucht zuerst gespeicherte Zugangsdaten. Wenn das fehlschlägt, startet ein lokales Setup-WLAN mit Konfigurationsportal.



```
#include <ESP8266WiFi.h>
#include <WiFiManager.h>

void setup() {
  Serial.begin(115200);

  WiFiManager wm;

  bool ok = wm.autoConnect("ESP8266-Setup");

  if (!ok) {
    Serial.println("WiFi failed");
    ESP.restart();
  }

  Serial.print("IP: ");
  Serial.println(WiFi.localIP());
}

void loop() {
}
```

- kein festes WLAN-Passwort im Sketch
- Konfiguration über Browser
- geeignet für Labor und Prototypen
- schnell in bestehende Arduino-Projekte integrierbar

Grenze

Das Portal ist komfortabel, aber nicht automatisch eine sichere Provisioning-Lösung.



```
#include <ESP8266WiFi.h>
#include <WiFiManager.h>

void setup() {
    Serial.begin(115200);

    WiFiManager wm;

    wm.setConfigPortalTimeout(180);

    bool ok = wm.autoConnect(
        "ESP8266-Setup",
        "setup-password"
    );

    if (!ok) {
        Serial.println("Provisioning timeout");
        ESP.restart();
    }

    Serial.println("WiFi connected");
}

void loop() {
}
```

Wichtige Punkte

- Setup-AP mit Passwort schützen
- Portal nur zeitlich begrenzen
- Fehlerfall definiert behandeln
- keine Zugangsdaten im Sketch

Grenze

WiFiManager trennt Zugangsdaten vom Code. Das Portal ist aber keine vollständige Sicherheitslösung.



- ESP-IDF bietet ein eigenes Provisioning-Framework.
- Zugangsdaten können nachträglich auf das Gerät übertragen werden.
- Unterstützte Wege sind z. B. SoftAP und Bluetooth LE.
- Transport, Protokoll und Security-Scheme sind getrennt.

Video: Espressif Unified Provisioning

Security-Schemes

- security0: keine Verschlüsselung, keine Authentifizierung
- security1: Curve25519 + AES-CTR, Proof of Possession
- security2: SRP6a + AES-GCM



SoftAP

- ESP startet ein eigenes Setup-WLAN.
- Nutzer verbindet sich mit diesem WLAN.
- Zugangsdaten werden über eine lokale Weboberfläche übertragen.
- Vorteil: funktioniert ohne zusätzliche Funktechnik.
- Nachteil: Nutzer muss kurz das WLAN wechseln.

BLE

- ESP wird über Bluetooth Low Energy gefunden.
- Einrichtung erfolgt typischerweise über eine App.
- Zugangsdaten werden über BLE übertragen.
- Vorteil: Nutzer bleibt in der App.
- Nachteil: benötigt BLE-Unterstützung und App-Logik.

Einordnung

SoftAP und BLE sind Transportwege für die Einrichtung. Die Absicherung der Zugangsdaten muss separat betrachtet werden.



Ansatz	Stärke	Typischer Einsatz
secrets.h	einfach	Labor, Übungen, Beispielcode
LittleFS-Datei	Config ohne Neukompilieren	lokale Konfiguration im Flash-Dateisystem
WiFiManager	Eingabe im Browser	Prototypen, wechselnde WLAN-Umgebungen
Espressif Provisioning	abgesichertes Provisioning	produktnahe Geräte, App-basierte Einrichtung

Praktische Auswahl

Für einfache Beispiele reicht `secrets.h`. Für komfortable Einrichtung ist `WiFiManager` geeignet. Für produktnahe Geräte ist abgesichertes Provisioning sinnvoll.



- ① Rückblick
- ② Grundlagen Wifi
- ③ WLAN-Zugangsdaten
- ④ ESP8266 als Webserver
- ⑤ Asynchroner Webserver

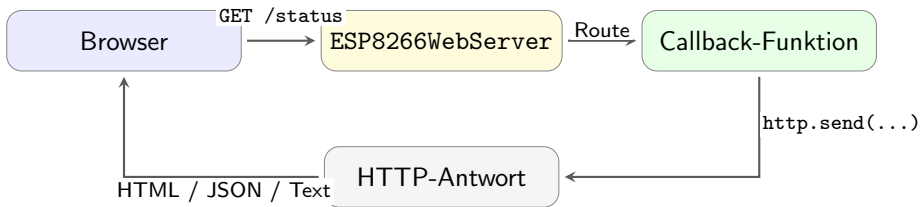


- Der ESP8266 kann selbst einen HTTP-Server bereitstellen.
- Ein Browser verbindet sich direkt mit der IP-Adresse des ESP8266.
- Der ESP verarbeitet eingehende Anfragen und erzeugt Antworten.
- Damit können Messwerte angezeigt und Aktoren gesteuert werden.

Einordnung

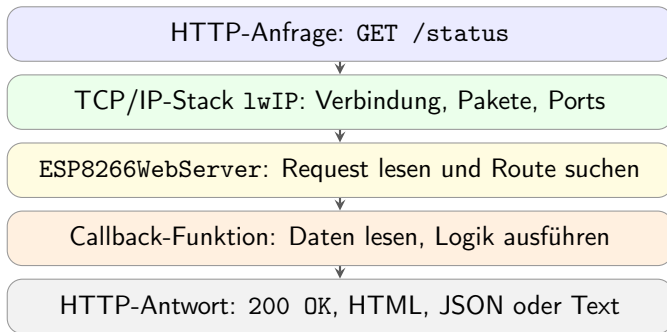
Der ESP8266 wird nicht nur Netzwerk-Client, sondern stellt selbst eine lokale Bedien- und Diagnoseschnittstelle bereit.

Request-Response-Prinzip auf dem ESP8266



Programmiermodell

Der Browser sendet eine Anfrage. Der ESP8266 ordnet die URL einer Callback-Funktion zu und sendet eine Antwort zurück.



Kernaussage

`http.handleClient()` verbindet Netzwerkstack und Anwendungscode: Es prüft auf neue Clients, liest den Request und ruft den passenden Handler auf.

Wichtig: Handler kurz halten. Lange blockierende Abläufe verzögern weitere Requests und Systemaufgaben.



```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>

ESP8266WebServer http(80);

void handleIndex() {
    http.send(200, "text/plain", "ESP8266 Webserver");
}

void setup() {
    Serial.begin(115200);

    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        delay(250);
    }

    http.on("/", handleIndex);
    http.begin();

    Serial.println(WiFi.localIP());
}

void loop() {
    http.handleClient();
}
```

- Port 80: normaler HTTP-Port
- `http.on(...)` registriert eine Route
- `http.begin()` startet den Server
- `http.handleClient()` bearbeitet Anfragen

Wichtig

`handleClient()` muss regelmäßig in der `loop()` aufgerufen werden.



```
void loop() {  
    updateWiFi();  
    http.handleClient();  
  
    readInputs();  
    updateOutputs();  
    updateStatusLed();  
}
```

Prinzip

`handleClient()` prüft eingehende HTTP-Anfragen, ruft den passenden Handler auf und sendet die Antwort.

Superloop-Regel

- jede Funktion läuft kurz
- keine endlosen Warteschleifen
- keine langen Handler
- Webinterface bleibt reaktionsfähig

Fehlerbild

Wenn der Webserver nur verzögert reagiert, liegt es oft an blockierendem Anwendungscode.



Route	Zweck
/	Startseite
/status	Diagnosewerte
/led	Ausgang schalten
/config	Einstellungen
/api/status	JSON-Daten

Prinzip

Die URL-Struktur ist die Schnittstelle zwischen Browser und Firmware.

```
void setupRoutes() {  
    http.on("/", handleIndex);  
    http.on("/status", handleStatus);  
    http.on("/led", handleLed);  
    http.on("/config", handleConfig);  
    http.on("/api/status", handleStatusJson);  
  
    http.onNotFound(handleNotFound);  
}  
  
void setup() {  
    // WiFi setup omitted  
  
    setupRoutes();  
    http.begin();  
}
```

Regel

Eine Route, eine klar benannte Handler-Funktion.



```
void handleStatus() {  
    String response;  
    response.reserve(160);  
  
    response += F("uptime_ms=");  
    response += millis();  
    response += F("\n");  
  
    response += F("ip=");  
    response += WiFi.localIP().toString();  
    response += F("\n");  
  
    response += F("rssi=");  
    response += WiFi.RSSI();  
    response += F("\n");  
  
    http.send(200, "text/plain", response);  
}
```

- einfache Diagnose
- im Browser direkt lesbar
- gut für frühe Entwicklungsphase
- reserve() reserviert Speicher vorab
- reserve() reduziert Heap-Fragmentierung
- F(...) hält konstante Textteile aus dem RAM heraus

Grenze

Dynamisch zusammengesetzte Antworten benötigen trotzdem RAM.



```
void handleIndex() {  
  String page;  
  page.reserve(512);  
  
  page += F("<!doctype html>");  
  page += F("<html>");  
  page += F("<head>");  
  page += F("<meta charset='utf-8'>");  
  page += F("<title>ESP8266</title>");  
  page += F("</head>");  
  page += F("<body>");  
  page += F("<h1>ESP8266 Webserver</h1>");  
  page += F("<p><a href='/status'>Status</a></p>");  
  page += F("<p><a href='/led?state=1'>LED an</a></p>");  
  page += F("<p><a href='/led?state=0'>LED aus</a></p>");  
  page += F("</body></html>");  
  
  http.send(200, "text/html", page);  
}
```

- schnell für kleine Seiten
- keine zusätzlichen Dateien nötig
- HTML liegt im Quellcode
- bei größeren Seiten unübersichtlich
- dynamisches String-Objekt belegt Heap

Einordnung

Für kurze Diagnose- und Testseiten verwendbar.



```
const char INDEX_HTML[] PROGMEM = R"rawliteral(
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <title>ESP8266</title>
</head>
<body>
  <h1>ESP8266 Webserver</h1>
  <p><a href="/status">Status</a></p>
  <p><a href="/led?state=1">LED an</a></p>
  <p><a href="/led?state=0">LED aus</a></p>
</body>
</html>
)rawliteral";

void handleIndex() {
  http.send_P(200, PSTR("text/html"), INDEX_HTML);
}
```

- PROGMEM: HTML liegt im Flash, nicht im RAM
- R"rawliteral(...)" ist ein Raw-String (Sonderzeichen)
- dadurch muss HTML kaum escaped werden
- PSTR("text/html") legt auch den MIME-Text im Flash ab
- send_P() sendet Daten direkt aus dem Programmspeicher

Regel

Statische HTML-Seiten in den Flash legen. Dynamische String-Erzeugung nur verwenden, wenn sich der Inhalt wirklich ändert.



```
const int LED_PIN = LED_BUILTIN;

void handleLed() {
  if (!http.hasArg("state")) {
    http.send(400, "text/plain", "missing state");
    return;
  }

  String state = http.arg("state");

  if (state == "1") {
    digitalWrite(LED_PIN, LOW); // built-in LED active low
    http.send(200, "text/plain", "LED on");
  } else if (state == "0") {
    digitalWrite(LED_PIN, HIGH);
    http.send(200, "text/plain", "LED off");
  } else {
    http.send(400, "text/plain", "invalid state");
  }
}
```

Aufruf im Browser

- /led?state=1
- /led?state=0

Prinzip

Parameter werden über
`http.arg(...)` gelesen.

Wichtig

Eingaben immer prüfen.
Browser-Parameter sind externe
Eingaben.



```
void redirectToIndex() {
    http.sendHeader("Location", "/");
    http.send(303, "text/plain", "");
}

void handleLed() {
    if (http.hasArg("state")) {
        int state = http.arg("state").toInt();

        if (state == 1) {
            digitalWrite(LED_PIN, LOW);
        } else {
            digitalWrite(LED_PIN, HIGH);
        }
    }

    redirectToIndex();
}
```

- Steuerroute führt Aktion aus
- danach Rücksprung auf Startseite
- Browser bleibt nicht auf technischer URL stehen
- Statuscode 303: siehe andere URL

Bedienlogik

Aktion ausführen, danach wieder Bedienoberfläche anzeigen.



```
float readTemperature() {
    return 23.4;
}

void handleStatusJson() {
    String json;
    json.reserve(160);

    json += F("{");
    json += F("\"uptime_ms\":");
    json += millis();
    json += F(",");

    json += F("\"temperature_c\":");
    json += readTemperature();
    json += F(",");

    json += F("\"rssi_dbm\":");
    json += WiFi.RSSI();

    json += F("}");

    http.send(200, "application/json", json);
}
```

- maschinell lesbar
- geeignet für JavaScript im Browser
- geeignet für Tests mit curl
- trennt Daten von Darstellung

Prinzip

HTML zeigt die Oberfläche.
JSON liefert die Daten.



```
void handleDashboard() {  
  http.send_P(  
    200,  
    PSTR("text/html"),  
    DASHBOARD_HTML  
  );  
}  
  
void setupRoutes() {  
  http.on("/", handleDashboard);  
  http.on("/api/status", handleStatusJson);  
}
```

```
async function updateStatus() {  
  const response = await fetch('/api/status');  
  const data = await response.json();  
  
  document.getElementById('status').textContent =  
    JSON.stringify(data, null, 2);  
}  
  
setInterval(updateStatus, 1000);
```

Prinzip

- / liefert die statische HTML-Seite
- /api/status liefert aktuelle Messwerte als JSON
- JavaScript ruft den JSON-Endpoint regelmäßig ab
- nur die Daten werden aktualisiert, nicht die ganze Seite

Trennung

HTML beschreibt die Oberfläche.
JSON liefert die aktuellen Werte.



Dateien im Projekt

```
data/  
|-- index.html  
|-- style.css  
'-- app.js
```

Prinzip

HTML, CSS und JavaScript liegen als Dateien im Flash-Dateisystem. Der Sketch enthält nur noch die Serverlogik.

```
#include <ESP8266WiFi.h>  
#include <ESP8266WebServer.h>  
#include <LittleFS.h>  
  
ESP8266WebServer http(80);  
  
void setup() {  
    Serial.begin(115200);  
  
    // WiFi setup omitted  
    if (!LittleFS.begin()) {  
        Serial.println("LittleFS failed");  
        return;  
    }  
  
    http.serveStatic(  
        "/",           // browser path  
        LittleFS,      // source filesystem  
        "/",           // root directory in LittleFS  
    ).setDefaultFile("index.html"); // default page for "/"  
  
    http.begin();  
}  
  
void loop() {  
    http.handleClient();  
}
```



```
void setupRoutes() {  
  http.serveStatic(  
    "/",  
    LittleFS,  
    "/index.html"  
  );  
  
  http.serveStatic(  
    "/style.css",  
    LittleFS,  
    "/style.css"  
  );  
  
  http.serveStatic(  
    "/app.js",  
    LittleFS,  
    "/app.js"  
  );  
  http.on("/api/status", handleStatusJson);  
}
```

- Dateien werden direkt aus LittleFS gelesen
- C++-Code bleibt kleiner
- Weboberfläche kann separat bearbeitet werden
- API-Endpunkte bleiben im Sketch

Prinzip

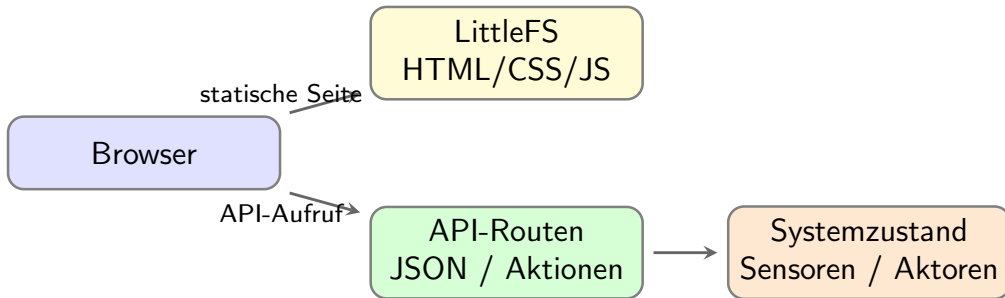
Statische Oberfläche aus Dateien, dynamische Daten über API.



Variante	Prinzip	Geeignet für
F("...") PROGMEM + send_P()	kurze konstante Texte im Flash statisches HTML aus dem Programmspeicher	Debugausgaben, kleine Meldungen kleine feste HTML-Seiten
LittleFS + serveStatic()	HTML, CSS und JS als Dateien im Flash	größere Weboberflächen
HTML + JSON-API	Oberfläche bleibt statisch, Werte kommen über /api/...	Dashboards mit aktualisierten Messwerten

Kernaussage

Statische Oberfläche getrennt halten. Dynamische Werte über JSON-Endpunkte nachladen.



Struktur

Oberfläche als statische Datei. Zustände und Aktionen als kleine API-Endpunkte.



```
void handleNotFound() {
    String message;
    message.reserve(160);

    message += F("404 Not Found\n\n");
    message += F("URI: ");
    message += http.uri();
    message += F("\nMethod: ");
    message += (http.method() == HTTP_GET) ? F("GET") : F("POST");
    message += F("\nArguments: ");
    message += http.args();

    http.send(404, "text/plain", message);
}

void setupRoutes() {
    http.on("/", handleIndex);
    http.on("/status", handleStatus);
    http.onNotFound(handleNotFound);
}
```

- falsche URLs sichtbar machen
- Parameter anzeigen
- Debugging erleichtern
- keine leeren Antworten erzeugen

Praxis

Ein sauberer Fehlerpfad spart Zeit bei Tests im Browser.



```
const char* WEB_USER = "admin";
const char* WEB_PASSWORD = "admin123";

bool checkAuth() {
    if (!http.authenticate(WEB_USER, WEB_PASSWORD)) {
        http.requestAuthentication();
        return false;
    }

    return true;
}

void handleProtectedLed() {
    if (!checkAuth()) {
        return;
    }

    handleLed();
}
```

- schützt gegen versehentliches Bedienen
- Browser zeigt Login-Dialog
- für Labor und lokale Tests ausreichend
- ohne HTTPS nicht gegen Mitschnitt geschützt

Grenze

Basic Auth über HTTP ist keine starke Sicherheitslösung.



- RAM ist begrenzt.
- Große dynamische String-Operationen vermeiden.
- Handler-Funktionen kurz halten.
- Keine langen Sensorabfragen direkt im Handler.
- `http.handleClient()` regelmäßig ausführen.
- Statische Inhalte aus Flash oder LittleFS ausliefern.
- Eingaben aus URL und Formularen immer prüfen.

Regel

Der Webserver darf die eigentliche Echtzeit- und Steuerlogik nicht blockieren.



- ① Rückblick
- ② Grundlagen Wifi
- ③ WLAN-Zugangsdaten
- ④ ESP8266 als Webserver
- ⑤ Asynchroner Webserver



- HTTP-Anfragen werden ereignisbasiert verarbeitet.
- Es gibt kein zyklisches `http.handleClient()` im `loop()`.
- Routen werden mit Callback-Funktionen registriert.
- Geeignet für Weboberflächen, JSON-APIs, Dateien, WebSocket und SSE.

Einordnung

Der Server reagiert auf Netzwerkereignisse. Der Anwendungscode sollte trotzdem kurz und nicht blockierend bleiben.



Aspekt	ESP8266WebServer	ESPAsyncWebServer
Request-Verarbeitung loop()	zyklisch über <code>handleClient()</code> muss Server regelmäßig bedienen	ereignisbasiert über Callbacks bleibt für Anwendungscode frei
API-Endpunkte	<code>/api/status</code> möglich	<code>/api/status</code> genauso möglich
JavaScript im Browser	<code>fetch()</code> bleibt gleich	<code>fetch()</code> bleibt gleich
Wichtig	nicht blockieren	Callbacks kurz halten

Merksatz

Die Webarchitektur ändert sich kaum. Nur die Serverlogik auf dem ESP wird anders organisiert.



```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>

const char* ssid = "ssid";
const char* password = "password";

AsyncWebServer server(80);

void setup() {
  Serial.begin(115200);

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  server.on("/", HTTP_GET,
    [](AsyncWebServerRequest* request) {
      request->send(200, "text/plain",
        "Hello from ESP8266");
    }
  );

  server.begin();
}

void loop() {
}
```

Auffällig

- kein `handleClient()`
- Route wird einmal registriert
- Callback bekommt ein request-Objekt
- Antwort über `request->send()`



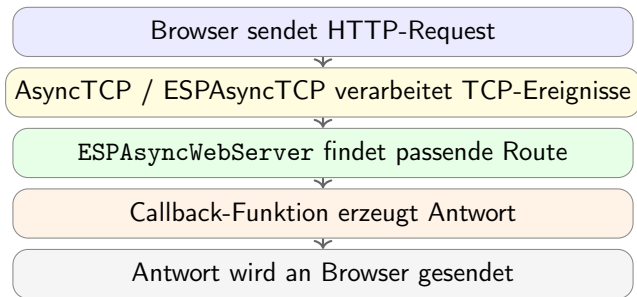
- ① Browser sendet einen HTTP-Request.
- ② Async-TCP-Schicht nimmt das Netzwerkereignis entgegen.
- ③ ESPAsyncWebServer sucht die passende Route.
- ④ Der registrierte Callback wird ausgeführt.
- ⑤ Die Antwort wird über das request-Objekt gesendet.

Wichtig

Asynchron bedeutet nicht, dass beliebig lange Berechnungen unkritisch sind. Callbacks müssen kurz bleiben.

Praxisregel

Keine langen `delay()`-Abschnitte und keine schweren Berechnungen im Handler.



Praxisregel

Callbacks kurz halten. Keine langen `delay()`-Blöcke, keine schweren Berechnungen im Handler.



```
void setupRoutes() {  
    server.on("/", HTTP_GET,  
        [] (AsyncWebServerRequest* request) {  
            request->send(200, "text/html",  
                "<h1>ESP8266</h1>");  
        }  
    );  
  
    server.on("/status", HTTP_GET,  
        [] (AsyncWebServerRequest* request) {  
            request->send(200, "text/plain", "OK");  
        }  
    );  
  
    server.onNotFound(  
        [] (AsyncWebServerRequest* request) {  
            request->send(404, "text/plain",  
                "Not found");  
        }  
    );  
}
```

Prinzip

- `server.on()` verbindet URL und Handler
- `HTTP_GET` legt die Methode fest
- `onNotFound()` behandelt unbekannte URLs

Regel

Eine Route, eine klare Aufgabe.



```
const int LED_PIN = 2;

void setupLedRoute() {
  server.on("/led", HTTP_GET,
    [](AsyncWebServerRequest* request) {

    if (!request->hasParam("state")) {
      request->send(400, "text/plain",
        "missing state");
      return;
    }

    String state =
      request->getParam("state")->value();

    digitalWrite(LED_PIN,
      state == "1" ? LOW : HIGH);

    request->send(200, "text/plain", "OK");
  }
);
}
```

Aufruf

/led?state=1

/led?state=0

- Parameter prüfen
- Wert auslesen
- Aktion ausführen
- kurze Antwort senden



```
float readTemperature() {  
    return 23.4;  
}  
  
void setupApiRoute() {  
    server.on("/api/status", HTTP_GET,  
        [] (AsyncWebServerRequest* request) {  
        String json;  
        json.reserve(128);  
  
        json += "{";  
        json += "\"uptime_ms\":";  
        json += millis();  
        json += ",";  
        json += "\"temperature_c\":";  
        json += readTemperature();  
        json += ",";  
        json += "\"rssi_dbm\":";  
        json += WiFi.RSSI();  
        json += "}";  
  
        request->send(200,  
            "application/json", json);  
        }  
    );  
}
```

Gleiches API-Prinzip

- Browser ruft
/api/status auf
- ESP liefert JSON
- JavaScript kann dieselbe
API verwenden
- Unterschied liegt nur im
Handler-Code



```
async function updateStatus() {  
  const response = await fetch('/api/status');  
  const data = await response.json();  
  
  document.getElementById('status').textContent =  
    JSON.stringify(data, null, 2);  
}  
  
setInterval(updateStatus, 1000);  
updateStatus();  
  
<pre id="status">loading...</pre>
```

Warum gleich?

- Der Browser kennt nur URLs.
- /api/status bleibt gleich.
- JSON-Format bleibt gleich.
- Nur die Serverbibliothek auf dem ESP ändert sich.

Trennung

Frontend und API bleiben stabil.
Die ESP-Implementierung kann wechseln.



- / liefert die statische Oberfläche.
- /api/status liefert Messwerte als JSON.
- /led?state=1 führt eine Aktion aus.
- Statische Dateien liegen in LittleFS.
- Handler bleiben kurz und senden klare HTTP-Statuscodes.

Vorteil

Der `loop()` muss den Webserver nicht aktiv pollen. Die Anwendung bleibt übersichtlicher.

Grenze

Asynchroner Server bedeutet nicht automatisch schnellerer Anwendungscode. Blockierende Handler bleiben ein Problem.



- Die Webserver-API ist auf ESP8266 und ESP32 sehr ähnlich.
- Für diese Folien wird der Code bewusst für den ESP8266 gezeigt.
- Auf dem ESP32 wird statt ESPAsyncTCP die Bibliothek AsyncTCP verwendet.
- Im Hintergrund unterscheiden sich Ressourcen, TCP-Anbindung und Tasking.

Praktische Aussage

Die Architektur bleibt gleich: statische Oberfläche, API-Routen, kurze Handler. Nur die Hardware- und Bibliotheksbasis darunter ist unterschiedlich.